Cours & Résumé — Projet Parc Véhicules

DEV AVANCE M1 — SQLAlchemy, Alembic, FastAPI

---

1. Architecture du projet

```
vehicule/
├── src/parc_vehicule/
│   ├── core/interfaces/        # Classes abstraites (ABC)
│   ├── models/                 # Modèles métier (Voiture, Thermique, Electrique,
Hybride)
│   ├── repositories/           # Accès BDD (raw SQL)
│   ├── routers/                # Routes FastAPI (cars.py, motors.py)
│   ├── services/               # Logique métier (Catalogue)
│   ├── database/               # Configuration SQLAlchemy
│   └── main.py                 # Point d'entrée FastAPI
├── alembic/                    # Migrations BDD
├── alembic.ini                 # Configuration Alembic
└── pyproject.toml              # Dépendances Poetry
```

---

2. SQLAlchemy

Qu'est-ce que c'est ?

SQLAlchemy est un ORM (Object Relational Mapper) Python. Il permet de communiquer avec une base de données via Python plutôt qu'en SQL brut.

Installation

```
poetry add sqlalchemy pymysql
```

- **sqlalchemy** — le framework ORM
- **pymysql** — le driver Python pour MySQL (remplace mysql-connector)

Configuration (`database/database.py`)

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base
import os
from dotenv import load_dotenv

load_dotenv()
```

```
user     = os.getenv("DB_USER",     "root")
password = os.getenv("DB_PASSWORD", "root")
host     = os.getenv("DB_HOST",     "localhost")
port     = int(os.getenv("DB_PORT", "3306"))
database = os.getenv("DB_NAME",     "projet_dev_avance")

SQLALCHEMY_DATABASE_URL = f"mysql+pymysql://{user}:{password}@{host}:
{port}/{database}"

engine = create_engine(SQLALCHEMY_DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Explication des éléments clés

Élément	Rôle
`create_engine(url)`	Crée la connexion à la BDD
`pool_pre_ping=True`	Vérifie que la connexion est vivante avant chaque requête
`SessionLocal`	Fabrique de sessions (une session = une transaction)
`Base`	Classe mère de tous les modèles ORM
`get_db()`	Générateur FastAPI — ouvre/ferme la session automatiquement

URL de connexion

```
mysql+pymysql://user:password@host:port/database
   ^       ^
   |       |
dialecte  driver
```

Deux modes d'utilisation de SQLAlchemy

Mode ORM — les modèles héritent de `Base` :

```
class Motor(Base):
    __tablename__ = "motors"
    id = Column(Integer, primary_key=True)
    type = Column(String(20))
```

Mode raw connection — utilisé dans ce projet pour compatibilité avec les repositories existants :

```
def get_connection():
    return engine.raw_connection()  # connexion pymysql brute
```

> Dans ce projet, on utilise `raw_connection()` pour ne pas réécrire tous les repositories qui utilisent du SQL brut avec curseur.

---

3. Variables d'environnement (.env)

Fichier `.env` à la racine du projet :

```
DB_HOST=localhost
DB_PORT=3306
DB_NAME=projet_dev_avance
DB_USER=root
DB_PASSWORD=root
```

Chargement avec `python-dotenv` :

```
from dotenv import load_dotenv
load_dotenv()
os.getenv("DB_USER", "root")  # "root" = valeur par défaut
```

> **Bonne pratique** : ne jamais hardcoder les credentials dans le code. Toujours utiliser des variables d'environnement.

---

4. Alembic — Migrations de base de données

Qu'est-ce que c'est ?

Alembic est un outil de migration de schéma de BDD pour SQLAlchemy. Il permet de versionner les modifications de la structure de la BDD (comme Git pour le code).

Installation

```
poetry add alembic
```

Initialisation

```
poetry run alembic init alembic
```

Crée :

- `alembic/` — dossier des migrations
- `alembic/env.py` — configuration de l'environnement
- `alembic/versions/` — fichiers de migration
- `alembic.ini` — configuration générale

Configuration de `alembic/env.py`

```python
import sys
from pathlib import Path
from dotenv import load_dotenv
from sqlalchemy import pool
from alembic import context

load_dotenv()
sys.path.insert(0, str(Path(__file__).resolve().parents[2] / "src"))

from parc_vehicule.database.database import Base, engine

config = context.config
target_metadata = Base.metadata  # pour l'autogenerate

def run_migrations_online():
    with engine.connect() as connection:
        context.configure(connection=connection, target_metadata=target_metadata)
        with context.begin_transaction():
            context.run_migrations()
```

Commandes essentielles

Commande	Description
`poetry run alembic revision -m "nom"`	Crée une migration vide
`poetry run alembic revision --autogenerate -m "nom"`	Crée une migration automatique depuis les modèles ORM
`poetry run alembic upgrade head`	Applique toutes les migrations en attente

Commande	Description
`poetry run alembic downgrade -1`	Annule la dernière migration
`poetry run alembic stamp <revision_id>`	Force le marqueur de version sans exécuter le SQL
`poetry run alembic current`	Affiche la version actuelle
`poetry run alembic history`	Affiche l'historique des migrations

Structure d'un fichier de migration

```
revision = 'ae12b7276c6b'       # ID de cette migration
down_revision = 'e41ed9c1d480'  # ID de la migration précédente

def upgrade() -> None:
    # Ce qui est exécuté lors de alembic upgrade
    op.create_table('motors', ...)

def downgrade() -> None:
    # Ce qui est exécuté lors de alembic downgrade
    op.drop_table('motors')
```

Migrations dans ce projet

Révision	Nom	Contenu
`e41ed9c1d480`	init	Migration neutre (pass)
`ae12b7276c6b`	init database	Création tables `motors` et `vehicles`
`e94bd9d6a70a`	seed data	Peugeot 308, Tesla Model 3, Volvo XC40
`09ffb7a6d448`	seed pagani bugatti cybertruck	Pagani Zonda, Bugatti Tourbillon, Tesla Cybertruck

Migration avec données (seed)

```
def upgrade() -> None:
    conn = op.get_bind()

    res = conn.execute(sa.text(
        "INSERT INTO motors (type, power, consumption_thermic, capacity_thermic) "
        "VALUES ('thermique', 130, 6.5, 50)"
    ))
    motor_id = res.lastrowid  # récupère l'id auto-incrémenté

    conn.execute(sa.text(
        f"INSERT INTO vehicles (brand, model, year, motor_id, ...) "
```

```
        f"VALUES ('Peugeot', '308', 2020, {motor_id}, ...)"
    ))
```

---

5. FastAPI — Routes corrigées

Route bonus : voitures d'une motorisation

```
GET /motorisations/{motorisation_id}/cars
```

Corrections apportées dans `motors.py` :

```
# AVANT (cassé)
from parc_vehicule.repositories.database_motor_repository import
DatabaseMotorRepository

voitures = DatabaseVehicleRepository(connection).get_by_motor_id(motorisation_id)
# ❌ DatabaseVehicleRepository non importé
# ❌ `connection` variable indéfinie

# APRÈS (corrigé)
from parc_vehicule.repositories.database_vehicle_repository import
DatabaseVehicleRepository
from parc_vehicule.routers.cars import VoitureRead
from parc_vehicule.routers.cars import vers_read as vers_voiture_read

voitures =
DatabaseVehicleRepository(get_connection()).get_by_motor_id(motorisation_id)
# ✅ import ajouté
# ✅ get_connection() utilisé
# ✅ alias vers_voiture_read pour éviter conflit de nom
```

Problème : doublon de route PUT

FastAPI interdit deux fonctions avec le même nom sur la même route. **Correction** : suppression du premier `PUT /{car_id}` qui contenait un `NotImplementedError`.

---

6. Lancer le projet

Prérequis

Fichier `.env` à la racine avec les credentials BDD.

Commandes

```
# Installer les dépendances
poetry install

# Appliquer les migrations (crée les tables + insère les données)
poetry run alembic upgrade head

# Lancer l'API
$env:PYTHONPATH="src"; poetry run uvicorn parc_vehicule.main:app --reload
```

Accès

- API : http://127.0.0.1:8000
- Documentation Swagger : http://127.0.0.1:8000/docs

---

7. Problèmes rencontrés et solutions

`ModuleNotFoundError: No module named 'parc_vehicule'`

Cause : `package-mode = false` dans `pyproject.toml`, le package n'est pas installé. **Solution** : définir `PYTHONPATH=src` avant de lancer uvicorn.

```
$env:PYTHONPATH="src"; poetry run uvicorn parc_vehicule.main:app --reload
```

`NameError: name 'Base' is not defined`

Cause : `Motorisation` héritait de `Base` (SQLAlchemy) sans l'importer. **Solution** : revenir à une classe ABC pure sans dépendance SQLAlchemy.

`Duplicate Operation ID` (warning FastAPI)

Cause : deux fonctions `update` avec `@router.put("/{car_id}")` dans `cars.py`. **Solution** : supprimer le doublon.

Migration Alembic supprime les tables

Cause : `--autogenerate` détecte les tables existantes comme "removed" car elles ne sont pas dans `Base.metadata`. **Solution** :

1. Corriger le `upgrade()` avec `pass` pour ne pas supprimer les tables
2. Utiliser `alembic stamp <id>` pour forcer le marqueur sans exécuter le SQL

---

8. Concepts clés à retenir

Pool de connexions

SQLAlchemy gère un pool de connexions automatiquement. `pool_pre_ping=True` envoie un `SELECT 1` avant chaque requête pour vérifier que la connexion est active.

raw_connection() vs Session ORM

- **Session ORM** : utilisée avec des modèles qui héritent de `Base`, gère les transactions automatiquement
- **raw_connection()** : connexion brute pymysql, compatible avec du SQL manuel via curseur

autogenerate Alembic

L'autogenerate ne fonctionne que si les modèles héritent de `Base`. Dans ce projet, les modèles sont des classes Python pures → les migrations doivent être écrites manuellement.

stamp vs upgrade/downgrade

- `upgrade` / `downgrade` : exécute le SQL de migration
- `stamp` : met à jour uniquement la table `alembic_version` sans exécuter de SQL — utile pour synchroniser l'état quand la BDD est déjà dans le bon état